Expenses Reporting Tool for an IT Consultancy

Project Management (GEP)
Assignment 2: Temporal Planning

Bachelor Degree in Informatics Engineering
Specialization in Software Engineering

Bachelor's Thesis

Natalia Yike Wang Yang

Thesis Supervisor
Sergio Alejandro Lobig Machado (FSP Consulting Services Limited)

Thesis Director
Marc Oriol Hilari (Department of Software Engineering)

GEP Tutor
Eduard Balbuena Longo (Department of Project Management)

Universitat Politècnica de Catalunya
Facultat Informàtica de Barcelona
Computer Science Department

March 5, 2026

Contents

5 Temporal Planning **1**
5.1 Task Description 1
5.1.1 Project Starting 1
5.1.2 Project Management 1
5.1.3 Design 2
5.1.4 Development 3
5.1.5 Frontend and Backend Development 3
5.1.6 Backend Development 5
5.1.7 Frontend Development 5
5.2 Resources 5
5.2.1 Human Resources 6
5.2.2 Physical Resources 6
5.3 Summary of Tasks and Resources 6
5.4 Gantt Diagram 9
5.5 Risks Management 11

List of Figures

5.4.1 Gantt Diagram (own creation) 10

List of Tables

5.2.1 Human Resources (own creation) 6
5.2.2 Hardware Resources (own creation) 6
5.2.3 Software Resources (own creation) 6
5.3.1 Project Tasks with Effort, Dependencies, and Resources (own creation) . . 9

5 Temporal Planning

This section introduces the initial project plan. The project begins on February 18, 2026, and ends on June 19, 2026, representing a total of 513 hours of effort over a period of 122 days. The main objective of this timeline is to achieve all the objectives defined in section 3.1.

According to Polytechnic University of Catalonia, the bachelor's thesis carries a workload of 18 ECTS credits, equivalent to 540 effort hours. However, the project completion date is set one week before the bachelor's thesis defense period, providing a buffer of 27 hours to accommodate potential setbacks.

5.1 Task Description

The key to achieving an effective time planning lies in analyzing all tasks, considering their priority and duration. This facilitates the sequencing of tasks and ensures the project is fully allocated.

All project tasks are categorized into five main groups: project starting, project management, design, development, and frontend/backend development. Each following subsection details all tasks in that category.

5.1.1 Project Starting

A solid project start requires a preliminary study and initial project definition. This aligns the project vision among all involved members and establishes a unified understanding of the project's scope and objectives. To achieve this, the following tasks are defined:

PS1. Context and Scope

This tasks covers the following sub-tasks:

- Problem and stakeholder identification
- Proposed solution
- Context and market study
- Scope definition (objectives, requirements, obstacles, and risks)
- Development methodology selection
- Monitoring and validation tools selection

PS2. Temporal Planning

Identify all tasks and resources, design a Gantt diagram, and define a risk management strategy.

PS3. Budget and Sustainability

Define a project budget and analyze the sustainability aspects of the proposed solution.

PS4. Initial Documentation

Consolidate the documentation from previous tasks into the definitive version of the initial documentation.

5.1.2 Project Management

As the name suggests, this section includes all tasks related to project management.

PM1. Agile Ceremonies

Run scrum daily meetings, sprint plannings, sprint reviews, and sprint retrospectives.

PM2. Thesis Director Review Meetings

Meet with the thesis director, allowing him to catch up and monitor project progress.

PM3. Git Repository Documentation

Maintain clear repository documentation, including an application setup guide and meaningful code comments.

PM4. User Manual Documentation

Write a user manual that serves as a reference guide for the application's end-users.

PM5. Product Backlog Update

Refine, update, and maintain the product backlog, ensuring it always reflects the current project status.

PM6. Final Thesis Documentation

Write the bachelor's thesis final documentation.

PM7. Final Demonstration Video

Record a final demonstration of the application once it is ready for use.

PM8. Bachelor's Thesis Defense

Plan and prepare a final exposition to defend and justify the bachelor's thesis achievements, while demonstrating commitment to the academic standards.

5.1.3 Design

This preliminary design phase establishes a clear application architecture to guide the subsequent implementation.

DT1. Use Case Diagram

Draw all use cases that the application offers to the user.

DT2. Conceptual Model Diagram

Design the database content, clarifying all entities, their attributes and the relationships between them.

DT3. Technology Stack Decision

Select the technology to be used for each application layer.

DT4. Logical and Physical Architecture Design

Design the logical architecture and describe how the implementation code is structured, including the purpose and role of each application layer. This task also includes designing the physical architecture, explaining how the application connects to external services and resources, both physical and virtual, and the function of each of them.

DT5. UI mockups Design

Design the overall style and appearance of the application.

DT6. Security Design

Design a secure user authentication, information submission, storage, and access control.

5.1.4 Development

This section includes essential tasks for application development.

Dev0. Self-learning and study of unfamiliar technologies

Study and learn new application technologies in advance.

Dev1. Environment Configuration

Configure all necessary environments, this includes installing the code editor Visual Studio Code, installing essential extensions, creating the Git repository and so on.

Dev2. Product Deployment

Release the final software product, allowing end users to access it when needed.

Dev3. Unit and Integration Testing

Include testing to verify that the implemented code functions correctly and to prevent coding bugs.

Dev4. Logging and Monitoring

Set up logging and monitoring tools to track application performance and review logs.

5.1.5 Frontend and Backend Development

As a full-stack web application, the implementation is divided into two main areas. The frontend is responsible for the user interface (UI) and user experience (UX). The backend implements the complex implementations hidden in the frontend and integrates external services via a RESTful API.

FBD1. Expense Uploads

Provide a visual interface in the frontend for users to drag-and-drop or upload receipts. The backend will later handle database updates to register the new ticket. In this way, uploaded expenses will be accessible on future logins.

FBD2. Uploaded Expenses Real-time Dashboard

Call the backend API from the frontend app for receiving all the previous uploaded receipts of the corresponding authenticated user. The frontend dashboard will display each expense and its real-time processing status, which can be:

- **Processing:** The expense is still being processed by the AI model.
- **Pending Review:** The AI model has successfully analyzed the expense, the uploader can now review it.
- **Completed:** The user has reviewed and completed all required information. The expense is ready for being included in a new or existing report.
- **Failed:** The AI couldn't extract data from the receipt (either due to unavailability or unrecognized format).

FBD3. AI-Extracted Expense Data Review and Correction

Let employees correct the AI-extracted expense data and fill in the missing required information in the frontend UI. Once the user wants to save the new expense information, the backend will then update the database with the corrected data.

FBD4. Uploaded Expenses Deletion

Allow employees to delete any uploaded receipt, regardless of its current status. The frontend provides visual tools to delete one or multiple expenses. After the user confirms the deletion, the backend marks them as deleted in the database.

FBD5. Expense Reports View, Creation, Edit, Submission, and Deletion

Allow employees to create reports from uploaded, AI-processed, and reviewed expenses. They can edit the report details, submit for approval, or delete unsubmitted reports using the frontend UI. The backend updates the database status based on user actions.

FBD6. Company Email User Authentication

Grant employees login access using their company email account through the frontend login page. The backend, which is integrated with Microsoft Entra ID, verifies the user access. On first login, the backend creates a new user profile; on return visits, it retrieves the user's relevant data (e.g. previous uploaded expenses).

FBD7. Role-based User Access

Assign the corresponding role to the logged-in user, either regular employee or administrator. Administrators have additional features unavailable to regular employees.

When a user logs into the frontend app, the backend retrieves their corresponding role, enabling the frontend to display only the functionalities specific to that role.

FBD8. Submitted Report Approval and Rejection

Enable administrators to view reports submitted by their supervised employees for approval or rejection of reimbursement budgets. The frontend handles the interface, while the backend updates the database accordingly.

FBD9. Employee Spending Limit Configuration

Provide administrative users the ability to set spending limits for employees under their supervision through the frontend UI. The backend is accountable for registering the new employee expense limit.

FBD10. Submitted and Reviewed Expenses Records Visualization

Enable users to view their submitted reports record with its corresponding current status. Administrators are also able to see their reviewed reports record, including their approval or rejection decision. To do that, the frontend calls the backend API to retrieve the accurate information and displays it to the user through a visual dashboard.

FBD11. Email Notification Service Integration

Send email notifications to the approver when there are new submitted reports pending to review, or to the employee when their report is approved or rejected. The backend handles all email notifications triggered by frontend actions.

5.1.6 Backend Development

The following tasks are implemented exclusively in the application backend logic.

BD1. RESTful API Backend Logic Implementation

Develop a RESTful API to manage backend responsibilities invoked by frontend requests.

BD2. Azure SQL Database Integration

Integrate Azure SQL Database with Entity Framework to manage and retrieve the relevant information.

BD3. Azure Blob Storage Integration

Integrate Azure Blob Storage to save original expense files.

BD4. Microsoft Document Intelligence AI Model Integration

Integrate an AI model to automate the expense data extraction.

BD5. Automated AI-driven Data Extraction Pipeline

Integrate Event Grid and Service Bus and implement a mechanized data extraction workflow. When a new receipt is uploaded and saved to Blob Storage and SQL Database, Azure Event Grid detects the event and routes it to Service Bus. This automates the AI model's processing, while preventing bottlenecks during multiple simultaneous uploads.

BD6. Azure SignalR Real-time Notification Service Integration

Integrate SignalR into the backend logic, enabling the backend to notify the frontend whenever the AI model processes a receipt. Once the expense extraction has succeeded or failed, users are notified immediately, enabling real-time UI updates without requiring a manual page refresh.

BD7. Microsoft Entra ID User Authentication Integration

Integrate Microsoft Entra ID into the backend logic to handle employee authentication through the company email account.

5.1.7 Frontend Development

This section covers all tasks that require only frontend logic implementation.

FD1. Expenses Search and Filter

Enable users to search uploaded expenses by name or filter them by category.

FD2. Reports Search

Enable users to search specific reports by name.

5.2 Resources

Following the analysis of all required tasks for successful project completion, all necessary resources must be identified. Resources are divided into two main groups: human resources and physical resources. All human resources are named in Table 5.2.1, specifying the responsible individual for each role. The physical resources are further categorized into software and hardware resources, and are identified and briefly described in Table 5.2.2 and 5.2.3.

5.2.1 Human Resources

Resource ID	Role Name	Responsible Individual
SM	Scrum Master	Thesis author
PO	Product Owner	Sergio Alejandro Lobig Machado
TD	Thesis Director	Marc Oriol Hilari
GT	GEP Tutor	Eduard Balbuena Longo
DT	Development Team	Thesis author

Table 5.2.1: Human Resources (own creation)

5.2.2 Physical Resources

Hardware Resources

Resource ID	Resource Name	Components
OS	Office Supplies	A table, a chair, electricity, wifi and office rental.
HW	Hardware Resources	A computer, two monitors, a mouse and a keyboard.

Table 5.2.2: Hardware Resources (own creation)

Software Resources

Resource ID	Resource Name	Description
GIT	Git	Version control tool for managing the source code repository.
AZR	Microsoft Azure Resources	Cloud services and resources: SQL Database, Blob Storage, AI model, Event Grid, Service Bus, SignalR, EntraID, and Azure DevOps (for repository and product backlog management).
MCS	Microsoft Collaboration Tools	Communication and coordination platforms: Microsoft Teams, Calendar, Outlook email, Word editor.
VSC	Visual Studio Code	Source code editor with extensions and debugging tools.
FIB	FIB Platforms	Platforms from Barcelona School of Informatics(FIB): Racó, Atenea, FIB secretary, bachelor's thesis official website guide.
GM	Gmail	Email communication with university entities.
WEB	Web Browsers and Search Engines	Research Browsers: Mozilla Firefox, Google Chrome, Microsoft Edge.
OVL	Overleaf LaTeX Editor	Thesis formatting editor for final thesis documentation.
OT	Online Tools	Diagram design tools: Edraw, Draw.io, and Gantt Project.

Table 5.2.3: Software Resources (own creation)

5.3 Summary of Tasks and Resources

Project Tasks Overview				
Task ID	**Task Name**	**Effort (h)**	**Dependen-cies**	**Resources**
PS	**Project Starting**	**60**		
PS1	Context and Scope	20	-	SM, PO, TD, GT, DT, OS, HW, MCS, VSC, FIB, GM, WEB, OVL
PS2	Temporal Planning	10	PS1	SM, PO, TD, GT, DT, OS, HW, MCS, VSC, FIB, GM, WEB, OVL, OT
PS3	Budget and Sustainability	10	PS2	SM, PO, TD, GT, DT, OS, HW, MCS, VSC, FIB, GM, WEB, OVL
PS4	Initial Documentation	10	PS3	SM, PO, TD, GT, DT, OS, HW, MCS, VSC, FIB, GM, WEB, OVL
PM	**Project Management**	**140**		
PM1	Agile Ceremonies	51	-	SM, PO, DT, OS, HW, AZR, MCS, VSC
PM2	Thesis Director Review Meetings	5	-	DT, TD, OS, HW, AZR, VSC, FIB, GM, WEB
PM3	Git Repository Documentation	40	Dev1	DT, OS, HW, GIT, AZR, VSC, WEB
PM4	User Manual Documentation	2	PM5	DT, OS, HW, VSC, WEB
PM5	Product Backlog Update	8	PS2	SM, DT, PO, OS, HW, AZR, WEB
PM6	Final Thesis Documentation	20	-	DT, TD, OS, HW, FIB, GM, WEB, OVL, OT
PM7	Final Demonstration Video	4	PM6	DT, OS, HW, WEB
PM8	Bachelor's Thesis Defense	20	PM6	DT, TD, OS, HW, FIB, GM, WEB
DT	**Design**	**16**		
DT1	Use Case Diagram	2	PS1	DT, OS, HW, OT, WEB, MCS
DT2	Conceptual Model Diagram	4	PS1	DT, OS, HW, OT, WEB
DT3	Technology Stack Decision	2	PS1	DT, OS, HW, WEB, MCS
DT4	Logical and Physical Architecture Design	4	PS1	DT, OS, HW, WEB, MCS
DT5	UI mockups Design	2	DT1	DT, OS, HW, OT, WEB
DT6	Security Design	2	DT3, DT4	DT, OS, HW, WEB, MCS
Dev	**Development**	**28**		
Dev0	Self-learning and study of unfamiliar technologies	8	DT3	DT, OS, HW, WEB
Dev1	Environment Configuration	2	DT2, DT5, DT6	DT, OS, HW, GIT, AZR, VSC, WEB

Table continues on the next page

Table 5.3.1 – continued from previous page

Task ID	Task Name	Effort (h)	Dependencies	Resources
Dev2	Product Deployment	8	Dev3	DT, OS, HW, GIT, AZR, VSC, WEB
Dev3	Unit and Integration Testing	10	FBD9, FBD10, FBD11	DT, OS, HW, GIT, AZR, VSC, WEB
Dev4	Logging and Monitoring	8	Dev2	DT, OS, HW, GIT, AZR, VSC, WEB
FBD	**Frontend and Backend Development**	**128**		
FBD1	Expense Uploads	16	Dev1	DT, OS, HW, GIT, AZR, VSC, WEB
FBD2	Uploaded Expenses Real-time Dashboard	4	FBD1, BD6	DT, OS, HW, GIT, AZR, VSC, WEB
FBD3	AI-Extracted Expense Data Review and Correction	20	FBD2	DT, OS, HW, GIT, AZR, VSC, WEB
FBD4	Uploaded Expenses Deletion	4	FBD2	DT, OS, HW, GIT, AZR, VSC, WEB
FBD5	Expense Reports View, Creation, Edit, Submission, and Deletion	20	FBD3, FBD4	DT, OS, HW, GIT, AZR, VSC, WEB
FBD6	Company Email User Authentication	8	BD7	DT, OS, HW, GIT, AZR, VSC, WEB
FBD7	Role-based User Access	16	FBD6	DT, OS, HW, GIT, AZR, VSC, WEB
FBD8	Submitted Report Approval and Rejection	8	FBD5, FBD7	DT, OS, HW, GIT, AZR, VSC, WEB
FBD9	Employee Spending Limit Configuration	8	FBD7	DT, OS, HW, GIT, AZR, VSC, WEB
FBD10	Submitted and Reviewed Expenses Record Visualization	4	FBD8	DT, OS, HW, GIT, AZR, VSC, WEB
FBD11	Email Notification Service Integration	20	FBD8	DT, OS, HW, GIT, AZR, VSC, WEB
BD	**Backend Development**	**120**		
BD1	RESTful API Backend Logic Implementation	20	Dev1	DT, OS, HW, GIT, AZR, VSC, WEB
BD2	Azure SQL Database Integration and Manipulation	20	BD1	DT, OS, HW, GIT, AZR, VSC, WEB
BD3	Azure Blob Storage Integration	8	BD2	DT, OS, HW, GIT, AZR, VSC, WEB
BD4	Microsoft Document Intelligence AI Model Integration	20	BD2, BD3	DT, OS, HW, GIT, AZR, VSC, WEB
BD5	Automated AI-driven Data Extraction Pipeline	20	BD4	DT, OS, HW, GIT, AZR, VSC, WEB

Table continues on the next page

Table 5.3.1 – continued from previous page

Task ID	Task Name	Effort (h)	Dependencies	Resources
BD6	Azure SignalR Real-time Notification Service Integration	16	BD5	DT, OS, HW, GIT, AZR, VSC, WEB
BD7	Microsoft Entra ID User Authentication Integration	16	BD1	DT, OS, HW, GIT, AZR, VSC, WEB
FD	**Frontend Development**	**10**		
FD1	Expenses Search and Filter	8	FBD2	DT, OS, HW, GIT, AZR, VSC, WEB
FD2	Reports Search	2	FBD5	DT, OS, HW, GIT, AZR, VSC, WEB

Table 5.3.1: Project Tasks with Effort, Dependencies, and Resources (own creation)

5.4 Gantt Diagram

A Gantt diagram is attached at the end of this section. Continuous arrows represent Finish-Start (FS) dependencies between tasks, while dashed arrows represent Finish-Finish (FF) dependencies. FS dependencies indicate that successor tasks cannot start until their predecessor tasks are completed. FF dependencies show that successor tasks cannot finish until their predecessor tasks are completed.

Referring to Diagram 5.4.1 , the project timeline spans 121 days. Considering the 502 total estimated hours, this equates to approximately 4 hours of work per day. Consequently, the weekly workload is 28 hours, matching the weekly effort allocated in the Gantt diagram.

Since the Agile Methodology is selected for this application development, each week in the Gantt diagram represents one sprint. To perform the Agile ceremonies described in Section 4.1, daily Scrums (15 minutes each) are scheduled every 2 days, resulting in 3 meetings per week that happen on every Monday, Wednesday, and Friday. Sprint planning, review, and retrospective require 45 minutes each. Thus, Agile ceremonies total 3 hours per week, which equals 51 hours over 17 weeks. Agile's iterative meeting structure leads the **PM1** task to span the entire Gantt diagram timeline.

Other project management tasks follow the same pattern:

- **PM2** remains undetermined due to undefined thesis director meetings, representing iterative appointments throughout the project lifetime.
- **PM3**, which depends on **Dev1** (beginning once the repository is created), spans most of the project duration to satisfy objective SG7 from Section 3.1, as clean code requires ongoing, extensive documentation.
- **PM5** starts after **PS2** (temporal planning) is completed and continues during the full application development, as the product backlog requires constant updates whenever new features or tasks are added.
- **PM6** runs over the project stage to ensure consistency with final deliverables.

Certain development tasks also demand additional time allocation. For example, **FBD1**, **FBD3**, and **FBD11**. **FBD1** is the first coding task; **FBD3** requires extensive coordination with frontend and backend; **FBD11** marks the author's first experience implementing email notifications. As a result, extra time is allocated due to likely challenges. **Backend Development** tasks similarly incur additional time costs. Finally, **FBD7** requires prior database employee hierarchy configuration, adding further time overhead.

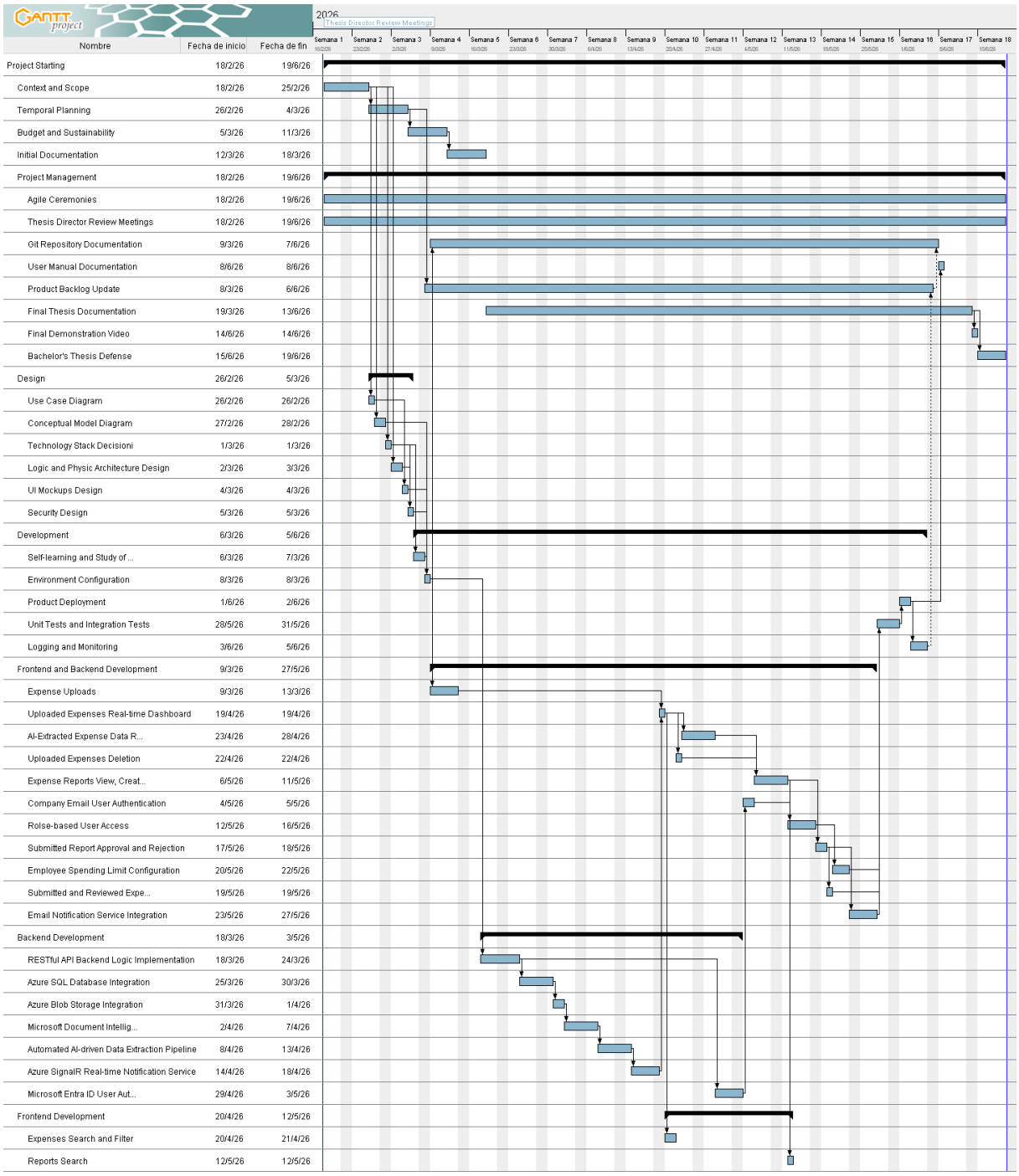

Figure 5.4.1: Gantt Diagram (own creation)

5.5 Risks Management

Possible risks affecting project progress are outlined in Section 3.3. With the timeline now established, contingency plans are defined to mitigate potential delays. This section presents each identified risk along with its probability, impact, and corresponding mitigation strategy.

Coding Bugs **Probability:** High

Impact: Medium

Coding bugs are inevitable, making thorough testing essential. As outlined in Section 5.4, every code change undergoes rigorous testing across all use cases, including edge cases. In addition, during the Dev1 task, Visual Studio Code (VS Code) extensions such as ESLint and SonarQube are installed to enable automated bug detection from the very beginning of implementation. These extensions highlight coding issues directly in the editor, preventing the developer from running the application until the errors are resolved.

If bugs persist, additional debugging time is allocated. However, thanks to the installed extensions and consistent testing, the number of bugs is minimized, reducing the likelihood of significant delays. Furthermore, complex tasks already include built-in buffer time, and the reserve hours are available to cover any remaining delays that may arise.

Tight Project Deadlines **Probability:** High

Impact: Low

Software development projects often struggle with tight deadlines. To overcome this challenge, the remaining 38 hours of overhead serve as a lifesaver. This can be particularly useful throughout the project, as unexpected delays are quite common.

Lack of Experience in Certain Technologies **Probability:** High

Impact: Medium

Due to the author's limited coding experience, several technologies used in the application implementation fall outside their core expertise. Task Dev0 addresses this challenge by allocating time for self-study, helping to minimize potential delays. If significant delays still persist, assistance from the FSP team serves as a fallback. Seeking timely assistance is an essential software development skill that helps avoid unnecessary deadline extensions.

Company's licenses and security permissions **Probability:** High

Impact: Low

Another risk involves FSP's security system. Cyberattacks are taken more seriously every day, often requiring developers to request and wait for permission to install resources and extensions. Consequently, new restrictions may limit initial resource availability. As a contingency, tasks with no pending dependencies, such as documentation, can be prioritized in such scenarios.